Harmonising TOGAF® with IAF

An enterprise architecture (EA) framework is for the architectural design and planning of large-scale changes to business systems.
A comprehensive framework should provide both processes and products.
This paper focuses on the terms and concepts used in defining products.
It draws some correspondences, and highlights some differences, between the two frameworks in the title.

Contents

The two frameworks

The generic relation that underpins TOGAF and IAF

Services and components

IAF templates for architecture artifacts

A service contract template for TOGAF

A flow description template for TOGAF

Data architecture

The two frameworks

Both frameworks were started in the mid 1990s, at the time when “object-oriented” methods were being superseded by “component-based” and “service-oriented” methods”.

The Open Group’s Architecture Framework (TOGAF) is a wide-ranging management framework for architecture-intensive work.
It is best known for its process, the Architecture Development Method (ADM).
Capgemini’s Integrated Architecture Framework (IAF) provides a process that is roughly comparable with the ADM

<table>
<tr><th>TOGAF’s ADM process</th><th colspan="2">IAF process</th></tr>
<tr><td rowspan="2">Vision</td><td>Vision</td><td></td></tr>
<tr><td>Strategy</td><td></td></tr>
<tr><td>Architecture Development</td><td rowspan="2">Design</td><td rowspan="3">Solution Architecture</td></tr>
<tr><td>Planning of Implementation</td></tr>
<tr><td>Governance of Implementation</td><td>Deliver</td></tr>
<tr><td>Governance of Change Management</td><td>Deploy</td><td></td></tr>
<tr><td></td><td>Retire</td><td></td></tr>
</table>

Architecture was first divided into Business, Data/Information, Applications and Technology domains in the PRISM report of 1986.
TOGAF offers a menu of products (deliverables and artifacts) for describing these domains.
IAF provides a set of template for describing architectural services and components across the same four domains.
“This holistic view of the business... is becoming even more critical as Service Oriented Architecture and the Service Oriented Enterprise become the way that many organisations think about and organise their business.” IAF 4.5

<table>
<tr><th>TOGAF domains</th><th colspan="5">IAF domains</th></tr>
<tr><td rowspan="2">Business</td><td rowspan="2">Business</td><td>Organisation and people</td><td rowspan="5">Security</td><td rowspan="5">Governance</td><td rowspan="5">Solution architecture</td></tr>
<tr><td>Services and processes</td></tr>
<tr><td>Data</td><td></td><td>Information</td></tr>
<tr><td>Applications</td><td rowspan="2">IT</td><td>Information systems</td></tr>
<tr><td>Technology</td><td>Technology infrastructure</td></tr>
</table>

IAF influenced the transition from version 8 to version 9 of the TOGAF standard.
IAF shares many concepts with TOGAF, but is not exactly the same and uses different terms.
This paper includes quotes from the version of IAF at the time it was mapped to TOGAF, which you can read at <https://agilearchitect.azurewebsites.net/useful_material/iaf/>

Note that both frameworks are hard to read, and questionable in places.
And both exclude the Software, Network and Storage aspects of IT Architecture.

The generic relation that underpins TOGAF and IAF

A simple relation underpins the artifacts used in TOGAF version 9 to define a business system.
The same relation is found in CapGemini's IAF.
This generic relation has been a little obscured by changes to the TOGAF standard in recent years.
Still, it makes sense; it enables TOGAF to be read as a coherent and consistent standard.
And knowing it helps readers recognise where some authors use core terms in different ways.

The table below expresses the generic relation in two different ways.
And shows the specialisation of that generic relation in the business, applications and technology domains, recorded using different artifacts.

Generic	Required behaviors	<clustered and assigned to>	Logical Active Structures	<realised by>	Physical Active Structures
TOGAF 9 artifacts	Services	<clustered and assigned to>	Logical Components	<realised by>	Physical Components
Business Service/Function catalog	**Business Services**		**Functions**		**Organization Units**
Role catalog + Actor/Role matrix	**Activities** (in processes)		**Roles**		**Actors**
Application portfolio catalog	**IS Services**		**Logical Application Components**		**Physical Application Components**
Technology portfolio catalog	**Technology Services**		**Logical Technology Components**		**Physical Technology Components**

The artifacts mentioned in the table above connect the three elements shown.
Other artifacts (some listed below) define other attributes and relationships of the same elements.

The service to component relationship
A service is *"a repeatable activity that a building block may be requested or otherwise triggered to perform."(TOGAF Ch. 3)*
TOGAF encourages architects to assign the responsibility for one service to one component.
And to minimise duplication of service provision by different components.
However, a component can delegate work to other components.

So, generally, one component may perform many services, and one service may be performed by many components.

The logical component to physical component relationship
In the application and technology domains, the ideal is a 1-to1 relation:

- one Logical Application Component is realised by one Physical Application Component.
- one Logical Technology Component is realised by one Physical Technology Component.

In practice, the relationships may be more complex, or logical components may be reverse-engineered to keep the relationship simple.
In the business domain, there are artifacts to capture the many-to-many logical-to-physical relationship .

Function/Organization matrix	N Functions	<are realised by>	N Organization Units
Actor/Role matrix	N Roles	<are realised by>	N Actors

Services and components

While the concepts of *service* and *component* in IAF and TOGAF are very similar, there are some critical differences.
And both frameworks sometimes use these terms with meanings that are, or seem, at odds with their standard meanings.

Term	IAF says	In TOGAF?
Service	An “element of behavior” or “unit of work”. (Beware IAF uses the term service where others might say *process*, or *flow* and oddly lists DBMS and Email *components* as examples of services.)	Similar “a repeatable activity that a building block may be requested or otherwise triggered to perform.
Service contract	See “Service description” in the next section.	Similar. See “Service contract” in the next section after next..
Business Service	An elementary unit of work a company does. All or part of the business may be described by Business Services and their Business Service Contracts.	Similar. Any system component is definable by the “bundle” or “portfolio” of services it can perform, with no regard to the internal procedures it follows to implement those services.
Component	A logical or physical entity that describes the structure of the architecture. IAF defines a component by the “List of services that are grouped in this component.” (Beware that IAF includes processes as a component type, even though processes are behavioral rather than structural.)	Similar, but services are not grouped “in” components. Rather, they appear in external interfaces to components.
Service implementation	In IAF, services interact with one or more “like services”. (This suggests IAF does not distinguish services from processes. TOGAF readers would presume components interact only by performing the processes that implement the services. Note also, the need for interacting services to be “like” each other questionable.)	Different A service is described with no indication of how a component performs it by interaction with services performed by other components (Services do interact indirectly, when the process that realizes a service invokes or responds to a process in another component, but those internal implementation details may change with no effect on the service.)

Building Block	*Services* are the fundamental building blocks of the architecture.	*Component*s are the fundamental building blocks of the architecture in some chapters. In other chapters, the term building block is used more loosely.

IAF templates for architecture artifacts

IAF provides templates for describing services, components and collaborations between them.
These templates are simplified, generalised and compared in the table below.
There are a few questionable inclusions and omissions, not discussed here.

SERVICE DESCRIPTION	COMPONENT DESCRIPTION	COLLABORATION CONTRACT
	List of services grouped in this component.	Consuming/calling Service OR Component name
	Type (Organizational/ Governance/ Process)	Owner/called Service OR Component name
Trigger/Actor	Trigger	
Confidentiality	Confidentiality	
Integrity	Integrity	
Availability	Availability	
Service window	Service window	Service window
Response time	Response time	Response requirements
Throughput	Throughput	Throughput
Throughput period	Throughput period	Throughput period
Scalability	Scalability	Growth
Scalability period	Scalability period	Growth period
MTTR	MTTR	
MTBF	MTBF	
Communication mechanism	Communication mechanism	
Quality of information required (preconditions)		Quality of information required
Quality of information delivered (post conditions)		
Result Input Object(s) Output Object(s) Error handling Input Control(s)	Component Quality Characteristics	Peak profile short term Peak profile long term Characteristics Contract control requirements Result control requirements Importance

Service descriptions as contracts
The universal characteristics of a service are signature, semantics and measures.
That is, it includes functionality (input/output flows and business rules) as well non-functional quality measures.
The functionality of a service appears at the bottom of the service description above.

Collaboration contracts
A collaboration contract defines a *flow* from a service to clients/consumers of that service.
(Note that in some cases, the requester, performer and consumer components of one service can be three different components.)
IAF defines a flow in terms of non-functional qualities only; it doesn't mention the payload of the flow.

"The *Service Collaboration Contract* specifies the content and characteristics of the relationship (aka interaction) between System Services." IAF
Note that this contract describe not the whole interaction, but only *a flow* from service provider to consumer.

The notion of a service-to-service collaboration is alien to a TOGAF reader who sees services as entries in external interfaces.
In any case, surely should this be called a component-to-service collaboration?
Since when component A requests a service from component B, it is usually irrelevant which service the component A itself is performing.

"*Component Collaboration Contracts* are aggregations of the Service Collaboration Contracts that already exist between Services." IAF
A component collaboration contract appears to cover *all the flows* one client component requires from one server component.

Defining point-to-point component collaboration contracts is more complex than TOGAF's concept of a "published interface" which is made available to all clients.
There is nothing in TOGAF about such contracts, but also, nothing to stop people building them out of service contracts and/or flow descriptions.

A service contract template for TOGAF

TOGAF includes business and application/IS service contracts in the Architecture Requirement Specification.
A service contract should define the functional and non-functional parameters for an interaction between client and server entities.
Those qualities may be included in, referred to in, or relate to measures including service level agreements (SLAs).

Generally, a service is from one server to any number of clients, meaning each client is offered the same contractual terms.
Suppose a client component wants to request (say) 5 of the 10 services performed by a server component?
Then the default assumption is that the component collaboration contract is based on the 5 general service contracts.

TOGAF 9.2 offers a vague and ambiguously defined "Contract/measure catalog" for service contract definition.
The best fit in IAF to TOGAF's service contract is IAF's service description template.
So, the template below is edited from that template.
Briefly:

- Service Name:
- Entry conditions: Trigger/Actor, Input flows, Preconditions, Error handling.
- Exit conditions: Output flows, Post conditions, Desired results.
- Measures: Response time: Throughput, Availability, Integrity, Security, Scalability.

At length:

Service Contract template	
Service Name:	A preferably unique short form name for the service, reflecting its goal.
KPIs:	A subset of the quality measures, E.g. · 98,5% delivered within defined business response times. · Max 5%of errors as a result of the business service. · Able to handle volumes up to 25% larger than the defined mean volumes.
Entry conditions Trigger/Actor: Inputs: Preconditions:	 The event or initiator that starts this service. What is consumed by this service, along with any control info required as input. The conditions necessary to assure successful and completion of the service. Error-handling procedures, signals, compensating actions.

Error handling:	
Exit conditions	
Outputs:	What is produced by this service, along with any control information provided as output.
Post conditions:	State changes at the end of successful processing.
Desired results:	The outcome(s) of successful service usage.
Communication	
Mechanism:	Face to face voice, Telephone voice, Mail, Fax, UI Click, eMail, SMS...
Automation:	Automated, Non-automated.
Style:	Fire and Forget, Request/Reply, File Transfer.
Non-functional quality measures	
Response time:	Normal/average and maximum time to complete a service.
Throughput:	Average number per <Second/ Minute/ Hour/ Day/ Week>.
Availability:	One of: Recoverable, Cold Standby, Hot standby, Fail safe (highest).
Service window	When the service should be available; opening hours.
Reliability:	MTBF of the service in case of incidents.
Recoverability:	MTTR of the service in case of incidents.
Integrity:	One of: Nominal, Standard, Individual, Double Intervention (highest).
Security:	Confidentiality: the level to which information should be protected from unauthorised disclosure or intelligible interception.
Scalability:	Maximum number of services per <Second/ Minute/ Hour/ Day/ Week>.

A flow description template for TOGAF

The service contract template above is a TOGAF-friendly version of IAF's service description template. The flow description template below is a TOGAF-friendly version of IAF's service collaboration template. The latter is needed to support the information exchange matrix and node connectivity diagram artifacts in phase B of TOGAF.

Briefly:

- Flow Name
- Sender and Receiver Names
- Content: The payload: the data or material structure transported and its format (unstructured, CSV, JSON, XML etc.)
- Measures: Response time: Throughput, Peak, Growth, Availability, Integrity, Security.

At length:

Flow description	
Sender name:	The entity sending the flow (in IAF, a service or component name).
Receiver name:	The entity receiving the flow (in IAF, a service or component name).
Content:	The payload: the data or material structure transported and its format (unstructured, CSV, JSON, XML etc.).
Response time:	Normal/average and maximum duration: < 1 Second, < 5 Seconds, > 5 seconds, 10 minutes, 1 hour, 1 day, 1 week.
Throughput:	Average number per Second/ Minute/ Hour/ Day/ Week.
Peak profile:	Peak number per Second/ Minute/ Hour/ Day/ Week.
Growth:	Percentage growth in a period of time.
Availability:	When the flow should be available.
Integrity:	Real time, maximum age of data, maximum error rate.
Security:	Confidentiality: the level to which information should be protected from unauthorised disclosure or intelligible interception.

Data architecture

IAF says: “Logical Information Components indicate potential data stores.”
TOGAF says: “The purpose of the Data Entity/Data Component catalog ... including data entities and also the data components where data entities are stored.”

In other words, an information/data component relates persistent data entities in a passive data structure that persists.
That data structure must survive the deletion of application/technology component instances, and the turning off of electricity to devices.
In the simple and normal case, one logical data component is realised in one physical data component, defined in the form required by a physical technology for storage in one location.
For some reasons (e.g. agile development) one logical data structure may be divided between two or more physical data components, which may be integrated asynchronously.
For other reasons, one physical data component may contain two or more logical data components.

IAF says	Closest corresponding TOGAF artifact
Information Object (catalog) Size: Describe the size of the information object. Numbers: Describes the number of information objects. Growth: Describe the expected growth in percentage.<%> Growth period: Describes the growth period.<Second/ Minute/ Hour/ Day/ Week/ Month/ Year> Confidentiality Classification: Classify and explain the level of Confidentiality to be supported by this component. Integrity Classification: Classify and explain the level of Integrity of this object. Availability Classification: Classify and explain the level of Availability of this object.	**Data Entity/Data Component catalog** This catalog is the definitive point of reference for data entities and data components in the information system domain. It defines persistent data entities (possibly with volumetric and security attributes) and relates them to data components they appear in. It is a base artifact for detailed data architecture. Note: A data entity can be structured or unstructured; it can be normalised or an aggregate. A data component encapsulates related data entities in a location. Where this catalog cannot include every data entity stored in every data component, it may record only core entities, especially ones that appear in several data components. Note that data flows are recorded in other artifacts.
Logical Information Component “describes the structure of Information Objects that supports the architecture solution. “describes Information Objects that have been grouped according to some grouping criteria. “Logical Information Components indicate potential data stores. “In some cases the Physical Information Components may be allocated to current state components (existing data stores for example).	**Logical Data Component** This relates data entities in a data structure that may be mapped to one physical data component, or divided between cohesively related data components.

Information system service description, includes “Input Object(s) consumed or used by this service “Output Object(s) created or transformed by this service or component.	**Contract/Measure catalog** (IS Service version)
Information Systems Information View Visualises how Information Objects are distributed across the Logical Information System Components.	**Data Dissemination Diagram** This diagram shows where selected data entities are disseminated (spread) across data components by application components. It may be drawn as a matrix. It may show data ownership, data replication, and master-copy relationships.
Business Service – Information Object Cross Reference A simple indication of whether the Business Service **uses, creates, or transforms** the Information Object.	**Data Entity/Business Function matrix**

Note that data flows are recorded in other artifacts.
Of course data entities can be captured and carried in data flows or messages (be they batch files, user interfaces or inputs/outputs in APIs or web services).
But those data flows are not data components in the sense defined in IAF or TOGAF above.
Moreover, what appears in data flows are often only primary keys, or partial data entities, or conflations of one data entity with another.
(The huge variety of partial and aggregate data entities cannot possibly be defined in the data entity/data component catalog.)

Data flows are weakly represented in most EA frameworks. However...
In IAF, data flow data structures are defined as “objects” in service descriptions.
In TOGAF, data flow structures are defined in or associated with the Interface Catalog and the Information Exchange Matrix.
For this reason, the artifact CRs include business and application data flow catalogs, in which the data flow structure is an attribute of a data flow.
Surely, rather than confuse the definition of data stores and data flows, the next step would be add a canonical data model for defining message elements?